

Boolean value. We can describe these settings as set (that is, `True`) or cleared (that is, `False`).

The false belief that flag arguments to function calls are bad is based on the claim that, depending on the flag value, the function does two entirely different things, such as in the following example:

```
def someFunction(flagArgument):  
    if flagArgument:  
        # Run some code...  
    else:  
        # Run some completely different code...
```

Indeed, if your function looks like this, you should create two separate functions rather than making an argument decide which half of the function's code to run. But most functions with flag arguments don't do this. For example, you can pass a Boolean value for the `sorted()` function's `reverse` keyword argument to determine the sort order. This code wouldn't be improved by splitting the function into two functions named `sorted()` and `reverseSorted()` (while also duplicating the amount of documentation required). So the idea that flag arguments are always bad is a code smell myth.

Myth: Global Variables Are Bad

Functions and methods are like mini-programs within your program: they contain code, including local variables that are forgotten when the function returns. This is similar to how a program's variables are forgotten after it terminates. Functions are isolated: their code either performs correctly or has a bug depending on the arguments passed when they're called.

But functions and methods that use global variables lose some of this helpful isolation. Every global variable you use in a function effectively becomes another input to the function, just like the arguments. More arguments mean more complexity, which in turn means a higher likelihood for bugs. If a bug manifests in a function due to a bad value in a global variable, that bad value could have been set anywhere in the program. To search for a likely cause of this bad value, you can't just analyze the code in the function or the line of code calling the function; you must look at the